Mammalia-Macaque-Dominance Network

Laura Silvana Alvarez Luque - Daniel Losada Molina

The aim of this project is to do a complete analysis of a network of interest. This includes: the network description, modeling through different techniques and results that can be obtained.

The selected network contains the dominance relations between *Macaca fuscata* females that were determined based on approach-retreat episodes around the food. The dominance range order was arranged based on these dyadic relations. Dominance is defined as "Physical contact unique to aggressive dominance interactions such as biting, head butting, fighting"

- **Vertices:** 62 Macaques.
- **Edges:** 1.167 edges representing the pairwise dominance between macaques.
- **Weights:** Frequency of the dominance.
- **Notes:** This is a animal interation network, undirected and weighted.

In [1]:
```
import zipfile
import base64
import random
import os

import numpy as np
import matplotlib.pyplot as plt
import igraph as ig
import plotly.graph_objects as go
import networkx as nx
import matplotlib.cm as cm
import matplotlib.colors as colors
import statsmodels.api as sm
from scipy.stats import expon, lognorm, poisson

from PIL import Image
from IPython.display import display
from collections import Counter
```

```
from igraph import *
```

Functions

In [2]:

```
def unzip_file(zip_path, extract_to):
    """
    Unzip a file from zip_path into the extract_to directory.
    """
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_to)

def get_node_info(g, node_id, threshold_pt=None):
    # Convert original ID to internal index
    internal_idx = g.vs.find(id=node_id).index

    if threshold_pt is not None:
        threshold = np.percentile(g.es["weight"], threshold_pt)
        print(f"Threshold for edge weight: {threshold}")

    incident_edges = g.incident(internal_idx, mode="ALL")
    count = 0
    node_strength = 0

    for eid in incident_edges:
        if threshold_pt is not None and g.es[eid]["weight"] < threshold:
            continue

        edge = g.es[eid]
        source = edge.source
        target = edge.target
        weight = edge["weight"]
        node_strength += weight

        source_id = g.vs[source]["id"]
        target_id = g.vs[target]["id"]
        print(f"Edge from {source_id} to {target_id}, weight = {weight}")
        count += 1

    print("Total incident edges (above threshold):", count)
    print(f"Total strength of node {node_id}:", node_strength)

def get_layout(g, layout_type="fr", seed=None):
    if seed is not None:
            np.random.seed(seed)
            np.random.seed(seed)
    if layout_type == "fr":
        print("Using Fruchterman-Reingold layout")
        return g.layout_fruchterman_reingold()
    elif layout_type == "kk":
        print("Using Kamada-Kaway layout")
        return g.layout_kamada_kawai()
    elif layout_type == "davidson_harel":
        print("Using Davidson-Harel layout")
        return g.layout_davidson_harel()
    elif layout_type == "graphopt":
        print("Using Graphopt layout")
        return g.layout_graphopt()
    else:
        return g.layout(layout_type)
```

```python
def plotly_graph(g):
    # Convert to NetworkX
    G_nx = g.to_networkx()

    # Get layout positions
    pos = nx.spring_layout(G_nx)

    # Prepare edge lines
    edge_x = []
    edge_y = []
    for u, v in G_nx.edges():
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_x += [x0, x1, None]
        edge_y += [y0, y1, None]

    edge_trace = go.Scatter(
        x=edge_x,
        y=edge_y,
        line=dict(width=0.5, color='#888'),
        hoverinfo='none',
        mode='lines',
        showlegend=False
    )

    # Create "hover points" at edge midpoints
    edge_hover_x = []
    edge_hover_y = []
    edge_text = []

    for u, v, data in G_nx.edges(data=True):
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_hover_x.append((x0 + x1) / 2)
        edge_hover_y.append((y0 + y1) / 2)
        weight = data.get("weight", 1)
        edge_text.append(f"{u} — {v}<br>weight: {weight}")

    edge_hover_trace = go.Scatter(
        x=edge_hover_x,
        y=edge_hover_y,
        mode='markers',
        hoverinfo='text',
        text=edge_text,
        marker=dict(size=5, color='rgba(0,0,0,0)'),  # invisible
        showlegend=False
    )

    # Prepare node positions
    node_x = []
    node_y = []
    node_text = []
    for node in G_nx.nodes():
        x, y = pos[node]
        node_x.append(x)
        node_y.append(y)
        node_text.append(f"Node {node}")

    node_trace = go.Scatter(
        x=node_x,
        y=node_y,
        mode='markers+text',
        text=[str(n) for n in G_nx.nodes()],
        textposition='top center',
```

```
        hoverinfo='text',
        marker=dict(
            size=10,
            color='lightblue',
            line_width=1
        )
    )

    # Combine everything
    fig = go.Figure(
        data=[edge_trace, edge_hover_trace, node_trace],
        layout=go.Layout(
            title='Dominance Network of Macaques',
            showlegend=False,
            hovermode='closest',
            margin=dict(b=20, l=5, r=5, t=40),
            xaxis=dict(showgrid=False, zeroline=False),
            yaxis=dict(showgrid=False, zeroline=False)
        )
    )

    fig.show()

def get_weights_of_path(g, path):
    """
    Given a path, return the weights of the edges along that path.
    """
    original_weights = []
    inv_weights = []

    for i in range(len(path) - 1):
        source = path[i]
        target = path[i + 1]

        # Get edge ID between consecutive vertices
        edge_id = g.get_eid(source, target)

        # Extract weights
        w = g.es[edge_id]["weight"]
        iw = g.es[edge_id]["inv_weight"]

        original_weights.append(w)
        inv_weights.append(iw)

    # Print results
    print("Original weights on path:", original_weights)
    print("Inverted weights on path:", inv_weights)


def plot_degree_distribution(g, title="Degree Distribution", weighted=False):
    """
    Computes degrees (weighted or unweighted), plots their distribution,
    and returns degree info including max degree and corresponding vertex/vertices.
    """
    # Choose between weighted and unweighted degree
    if weighted:
        degrees = g.strength(weights=g.es["weight"])
    else:
        degrees = g.degree()

    # Compute degree distribution
    unique_degrees, counts = np.unique(degrees, return_counts=True)
    probabilities = counts / counts.sum()
```

```
    # Plot the distribution
    plt.figure(figsize=(8, 5))
    plt.scatter(unique_degrees, probabilities, color='black')  # Dots
    plt.vlines(unique_degrees, 0, probabilities, color='blue', alpha=0.6)  # Lines

    # Labels and title
    plt.xlabel("Degree")
    plt.ylabel("Frequency")
    plt.title(title)

    # Show plot
    plt.show()

    # Get max degree and vertex/vertices
    max_degree = max(degrees)
    #vertices_with_max = [v.index for v, d in zip(g.vs, degrees) if d == max_degree
    vertices_with_max = [v['id'] for v, d in zip(g.vs, degrees) if d == max_degree]

    # Return degrees and distribution if needed later
    return degrees, unique_degrees, probabilities, max_degree, vertices_with_max

# Network Visualization Funcitons

def plot_simple_graph(g, layout_type="fr", target=None, display_img=True):

    if target is not None and os.path.exists(target):
        print(f"[✓] Skipping {target} (already exists)")
        if display_img:
            img = Image.open(target)
            display(img)
        return

    # Layout for node positions
    layout = get_layout(g,layout_type, 253)
    vertex_label=[str(i) for i in g.vs["id"]]

    # Plot using igraph's built-in plot function
    plot_obj = ig.plot(
        g,
        layout=layout,
        vertex_label=vertex_label,  # show node indices
        edge_width=[w for w in g.es["weight"]],
        bbox=(800, 800),
        margin=50,
        target=target
    )

    if display_img:
        display(plot_obj)


def plot_weighted_graph(g, layout_type="fr", target=None, display_img=True):

    if target is not None and os.path.exists(target):
        print(f"[✓] Skipping {target} (already exists)")
        if display_img:
            img = Image.open(target)
            display(img)
        return
    g_copy = g.copy()
    # Layout for node positions
    layout = get_layout(g_copy,layout_type, 253)

    weights = np.array(g_copy.es["weight"])
```

```
    threshold = np.percentile(weights, 70)
    norm = colors.Normalize(vmin=weights.min(), vmax=weights.max())

    # Compute node strength from strong edges
    node_strength = np.zeros(g_copy.vcount())
    for e in g_copy.es:
        if e["weight"] >= threshold:
            node_strength[e.source] += e["weight"]
            node_strength[e.target] += e["weight"]

    # Normalize node strength
    min_size, max_size = 20, 80  # Adjust for igraph scale
    normalized_strength = (node_strength - node_strength.min()) / (node_strength.ma
    node_sizes = min_size + (max_size - min_size) * normalized_strength
    node_colors = [colors.to_hex(cmap(val)) for val in normalized_strength]  # Conv

    # Set vertex attributes
    g_copy.vs["size"] = node_sizes
    g_copy.vs["color"] = node_colors
    g_copy.vs["label"] = g_copy.vs["name"]
    g_copy.vs["label_size"] = 10

    # Set edge attributes
    for e in g_copy.es:
        w = e["weight"]
        if w >= threshold:
            e["color"] = colors.to_hex(cmap(norm(w)))
            e["width"] = 1.5
            e["alpha"] = 0.6
        else:
            e["color"] = "#D3D3D3"  # Light gray for weak edges
            e["width"] = 0.3
            e["alpha"] = 0.2

    # Plot using igraph's built-in plot function
    plot_obj = ig.plot(
        g_copy,
        layout=layout,
        edge_color=g_copy.es["color"],
        edge_width=g_copy.es["width"],
        vertex_color=g_copy.vs["color"],
        vertex_size=g_copy.vs["size"],
        vertex_label=g_copy.vs["label"],
        vertex_label_size=g_copy.vs["label_size"],
        bbox=(800, 800),
        margin=50,
        target=target
    )

    if display_img:
        display(plot_obj)
```

In [3]:

```
%%capture
def plotly_graph_with_same_image(g, image_path='./imgs/macaco_agua_sin_fondo.png'):
    # Load and encode image as base64
    with open(image_path, "rb") as image_file:
        encoded_image = base64.b64encode(image_file.read()).decode()
        image_uri = f'{encoded_image}'

    # Convert to NetworkX
    G_nx = g.to_networkx()
    pos = nx.spring_layout(G_nx)

    # Prepare edge lines
```

```
    edge_x = []
    edge_y = []
    for u, v in G_nx.edges():
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_x += [x0, x1, None]
        edge_y += [y0, y1, None]

    edge_trace = go.Scatter(
        x=edge_x, y=edge_y,
        line=dict(width=0.5, color='#888'),
        hoverinfo='none', mode='lines',
        showlegend=False
    )

    # Prepare node positions and hover info
    node_x = []
    node_y = []
    node_text = []
    for node in G_nx.nodes():
        x, y = pos[node]
        node_x.append(x)
        node_y.append(y)
        node_text.append(f"Node {node}")

    node_trace = go.Scatter(
        x=node_x, y=node_y,
        mode='markers',
        hoverinfo='text',
        text=node_text,
        marker=dict(size=20, color='rgba(0,0,0,0)'),  # invisible
        showlegend=False
    )

    label_trace = go.Scatter(
        x=node_x,
        y=[y - 0.03 for y in node_y],
        mode='text',
        text=[str(n) for n in G_nx.nodes()],
        textposition='bottom center',
        hoverinfo='none',
        showlegend=False
    )

    # Add the same image to all nodes
    image_traces = []
    for x, y in pos.values():
        image_traces.append(
            dict(
                source=image_uri,
                xref="x", yref="y",
                x=x - 0.05, y=y + 0.05,
                sizex=0.1, sizey=0.1,
                xanchor="left", yanchor="top",
                sizing="stretch",
                opacity=1,
                layer="above"
            )
        )

    # Build the figure
    fig = go.Figure(
        data=[edge_trace, node_trace, label_trace],
        layout=go.Layout(
```

```
            title='Graph with Macaque Image on Each Node',
            images=image_traces,
            showlegend=False,
            hovermode='closest',
            margin=dict(b=20, l=5, r=5, t=40),
            xaxis=dict(showgrid=False, zeroline=False),
            yaxis=dict(showgrid=False, zeroline=False)
        )
    )

    fig.show()
```

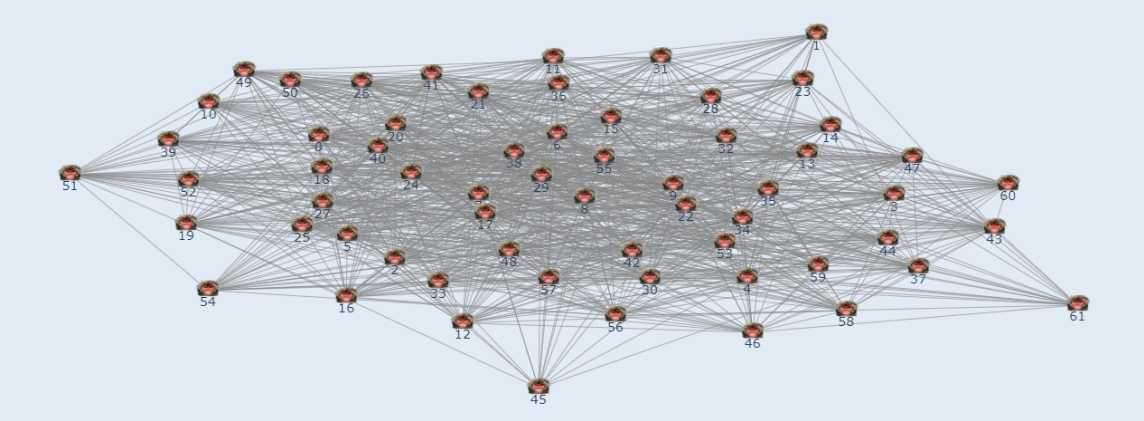
1. Input data

In [4]:
```
unzip_file('./mammalia-macaque-dominance.zip', './data')
```

In [5]:
```
#%%capture
g = ig.Graph.Read_Ncol("data/mammalia-macaque-dominance.edges", weights=True, direc
g.vs["id"] = [int(name) for name in g.vs["name"]]
#g.vs.find(id=4).index
print(g.summary())
```

```
IGRAPH UNW- 62 1167 --
+ attr: id (v), name (v), weight (e)
```

In [6]:
```
%%capture
#plotly_graph(g)
```



In [7]:
```
order = g.vcount()
size = g.ecount()

print(f"The order of the network is: {order} (vertices)")
print(f"The size of the network is: {size} (edges)")
```

```
The order of the network is: 62 (vertices)
The size of the network is: 1167 (edges)
```

We are dealing with a highly connected graph, as we can see by the edge-vertex ratio:

$$\frac{|E|}{|V|} = \frac{1167}{62} \approx 18.82$$

This means that in average, we have 18.82 edges per vertex. That's why is not surprising to find that we only have one big component that contains all vertices. Also for this reason, we don't have any subnetwork of interest and all the analysis will be done in the whole network.

In [8]:
```
components = g.components()
print(f"Number of subnetworks: {len(components)}")
```

```
Number of subnetworks: 1
```

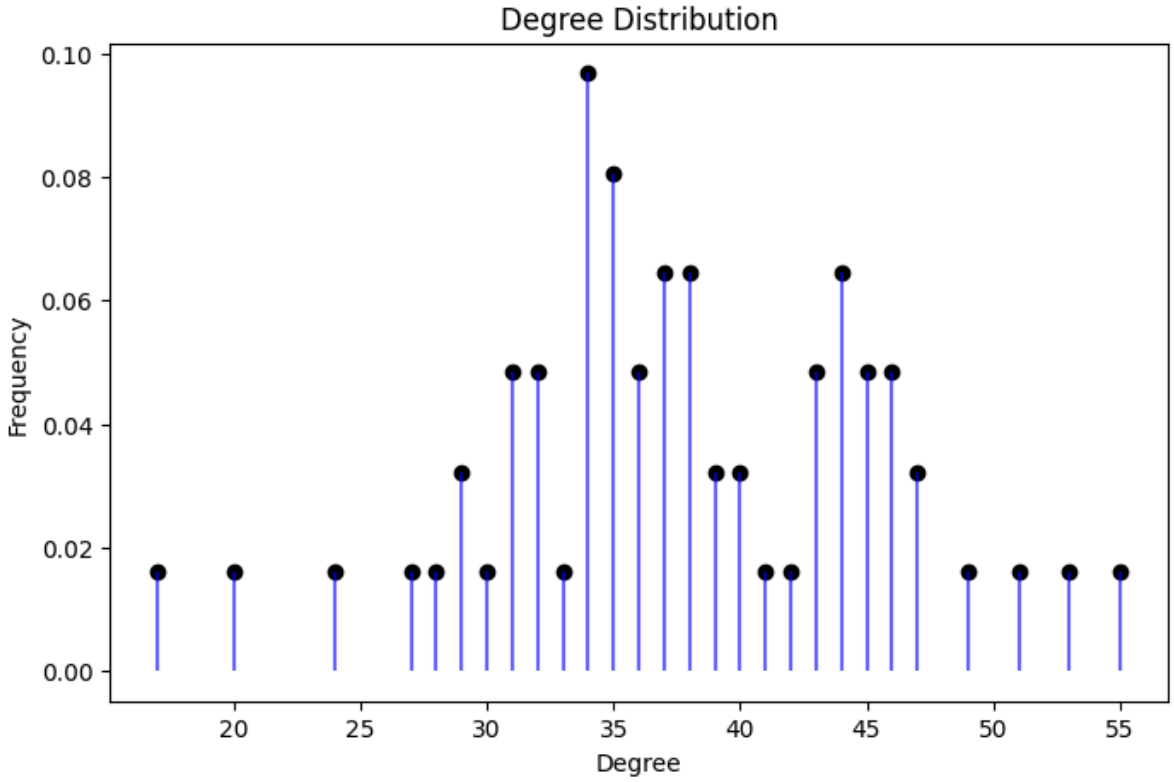
Due to the nature of our data, loops make no sense, as the edges represent interaction between two individuals. Anyways we check that our data has zero edges with same source and target.

In [9]:
```
sum(g.is_loop())
```

Out[9]:
```
0
```

With the degree distribution we can check that the network is highly connected as we saw previously. It does not have the clasical decreasing distribution starting in small degree values, instead, it starts in 14 and seems to have a bimodal distribution with concentrations around 34 and 44. Also, the most conected vertex is the 29 with 55 edges connected to it.

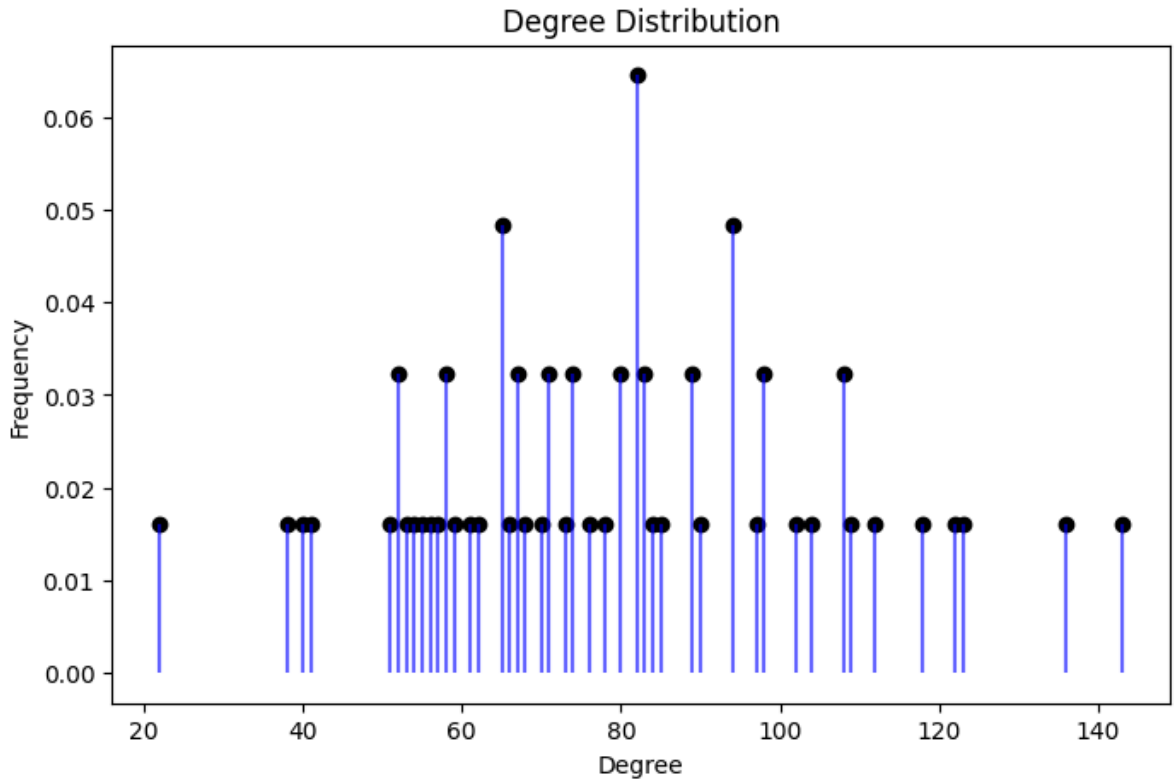
In [10]:
```
degrees, unique_deg, probs, max_deg, max_vertices = plot_degree_distribution(g)

print(f"Max degree: {max_deg}")
print(f"Vertices with max degree: {max_vertices}")
```




```
Max degree: 55
Vertices with max degree: [48]
```

Analysing the weighted degree distribution, the bimodal distribution is lost, and the maximum is centered around 80. On the other hand, the minimum value remains large, and the most connected vertex is still the 29.

In [11]:
```
degrees_w, unique_deg_w, probs_w, max_deg_w, max_vertices_w = plot_degree_distribut

print(f"Max degree: {max_deg_w}")
print(f"Vertices with max degree: {max_vertices_w}")
```




```
Max degree: 143.0
Vertices with max degree: [48]
```

Next we are going to compute the diameter.

In [12]:
```python
# Diameter (unweighted)
diameter = g.diameter(directed=False, weights=None)
print("Unweighted diameter:", diameter)

# Get the actual pair of farthest nodes and the path
farthest_pair_idx = g.get_diameter(directed=False, weights=None)
farthest_path_names = [g.vs[i]["id"] for i in farthest_pair_idx]

print("Farthest vertices (unweighted) igraph idx:", farthest_pair_idx)
print("Farthest vertices (unweighted original IDs):", farthest_path_names)
```

```
Unweighted diameter: 2
Farthest vertices (unweighted) igraph idx: [0, 1, 39]
Farthest vertices (unweighted original IDs): [1, 2, 29]
```

As we already seen, our graph is highly connected, so that the unweighted diameter is 2 is quite intuitive. That means that between two indivituals, there is a path of at most 2 (another individual in the middle). But it's not a special case to have a shortest distance of 2, in fact we have 1448 paths of length 2. This means that there are 1448 pairs of individuals that are not directly connected, but they are connected through another individual.

In [13]:
```python
sp_lengths = g.distances(weights=None)  # or .shortest_paths()

distances = [dist for row in sp_lengths for dist in row if dist != 0]
dist_count = Counter(distances)
print(dist_count)
```

```
Counter({1: 2334, 2: 1448})
```

To compute the weighted diameter, we need to invert the weights, as in our case higher weight means higher connection.

In [14]:
```
# Add inverse weights (handle division by zero if needed)
g.es["inv_weight"] = [1/w if w != 0 else float("inf") for w in g.es["weight"]]

# Then compute diameter using this new attribute
diameter_w = g.diameter(directed=False, weights="inv_weight")
path_w_idx = g.get_diameter(directed=False, weights="inv_weight")
path_w_names = [g.vs[i]["id"] for i in path_w_idx]

print("Weighted diameter (based on interaction):", diameter_w)
print("Farthest vertices (based on interaction) igraph idx:", path_w_idx)
print("Farthest path (based on interaction original IDs):", path_w_names)
```

```
Weighted diameter (based on interaction): 1.083333333333333
Farthest vertices (based on interaction) igraph idx: [45, 43, 56, 54]
Farthest path (based on interaction original IDs): [6, 61, 60, 46]
```

The longest shortest path is connected through a path whose total inverse-weight is 1.0833. Notice that using the weights now we obtain a farthest path that contains 3 edges that connect 4 individuals.

In [15]:
```
get_weights_of_path(g, path_w_idx)
```

```
Original weights on path: [3.0, 2.0, 4.0]
Inverted weights on path: [0.3333333333333333, 0.5, 0.25]
```

Finally, we compute the adjancency matrix and compare the results previously obtained.

In [16]:
```
# Get the adjacency matrix
adj_matrix = np.array(g.get_adjacency().data)
adj_matrix_w = np.array(g.get_adjacency(attribute="weight").data)

# Print the adjacency matrix
print(adj_matrix)
print(adj_matrix_w)
```

```
[[0 1 1 ... 0 0 0]
 [1 0 0 ... 0 0 0]
 [1 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 1 0]
 [0 0 0 ... 1 0 0]
 [0 0 0 ... 0 0 0]]
[[0. 3. 1. ... 0. 0. 0.]
 [3. 0. 0. ... 0. 0. 0.]
 [1. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 4. 0.]
 [0. 0. 0. ... 4. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]
```

In [17]:
```
# Results from the adjacence matrix

## degrees
degrees_adj = adj_matrix.sum(axis=0)

## directed or undirected
is_symmetric = np.allclose(adj_matrix, adj_matrix.T)

## Loops
self_loops = np.trace(adj_matrix)

## Nodes with degree 0 (isolated)
isolated = np.where(adj_matrix.sum(axis=1) == 0)[0]
```

```
print(degrees_adj)
print(is_symmetric)
print(self_loops)
print(isolated)
```

```
[38 24 44 36 38 41 49 46 46 47 31 38 34 35 36 42 35 53 40 35 45 39 35 32
 47 40 35 44 33 55 43 32 39 31 45 43 37 32 51 34 46 37 45 37 34 17 29 38
 44 34 31 30 34 43 27 36 37 44 29 34 28 20]
True
0
[]
```

The density is defined as:

$$Density = \frac{2 \times edges}{n(n-1)}$$

So is making a relation between the connections and the number of vertices. A high number means a dense network, while a small number a sparse one.

In [18]:
```
# Compute and print density
density = g.density(loops=False)
print(f"\nNetwork density: {density:.4f}")

# Interpret
if density > 0.5:
    print("This is a dense network.")
elif density < 0.1:
    print("This is a sparse network.")
else:
    print("This is a moderately connected network.")
```

```
Network density: 0.6171
This is a dense network.
```

Network Visualization

Energy Function

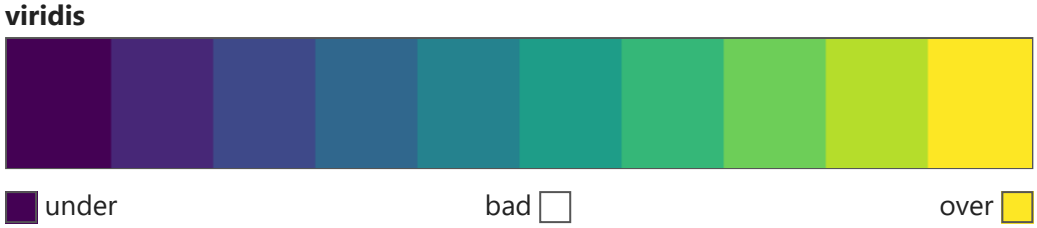
Davidson-Harel layout

For displaying the networks, the next color gradient is going to be used to represent the weigths. Also, more important vertex are displayed bigger and the edges with weights less than 2, are displayed in light gray in order to avoid a more crowded plot.

In [19]:
```
cmap = cm.get_cmap('viridis', 10)
cmap
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_26752\3613669891.py:1: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap('viridis', 10)
```

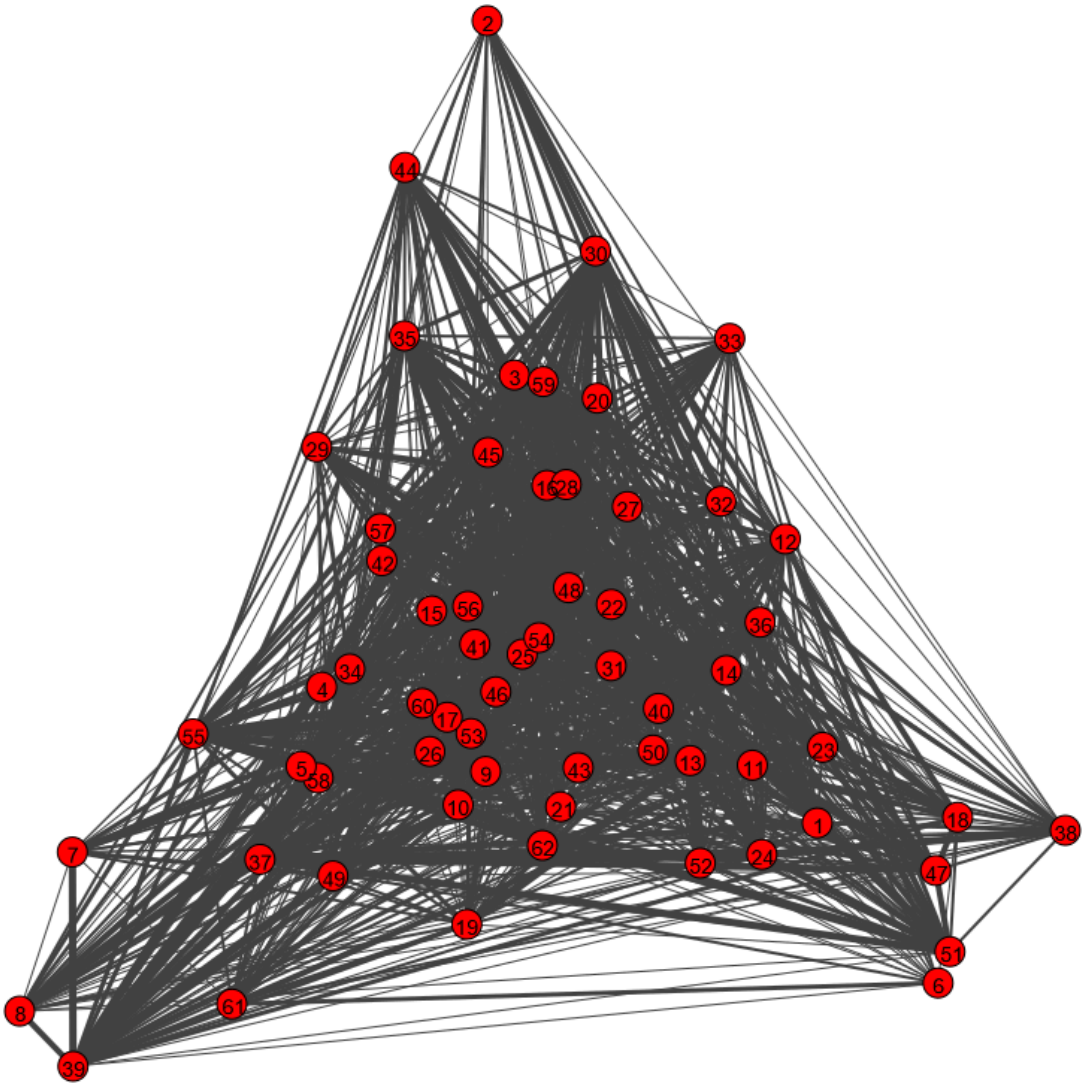
Out[19]:



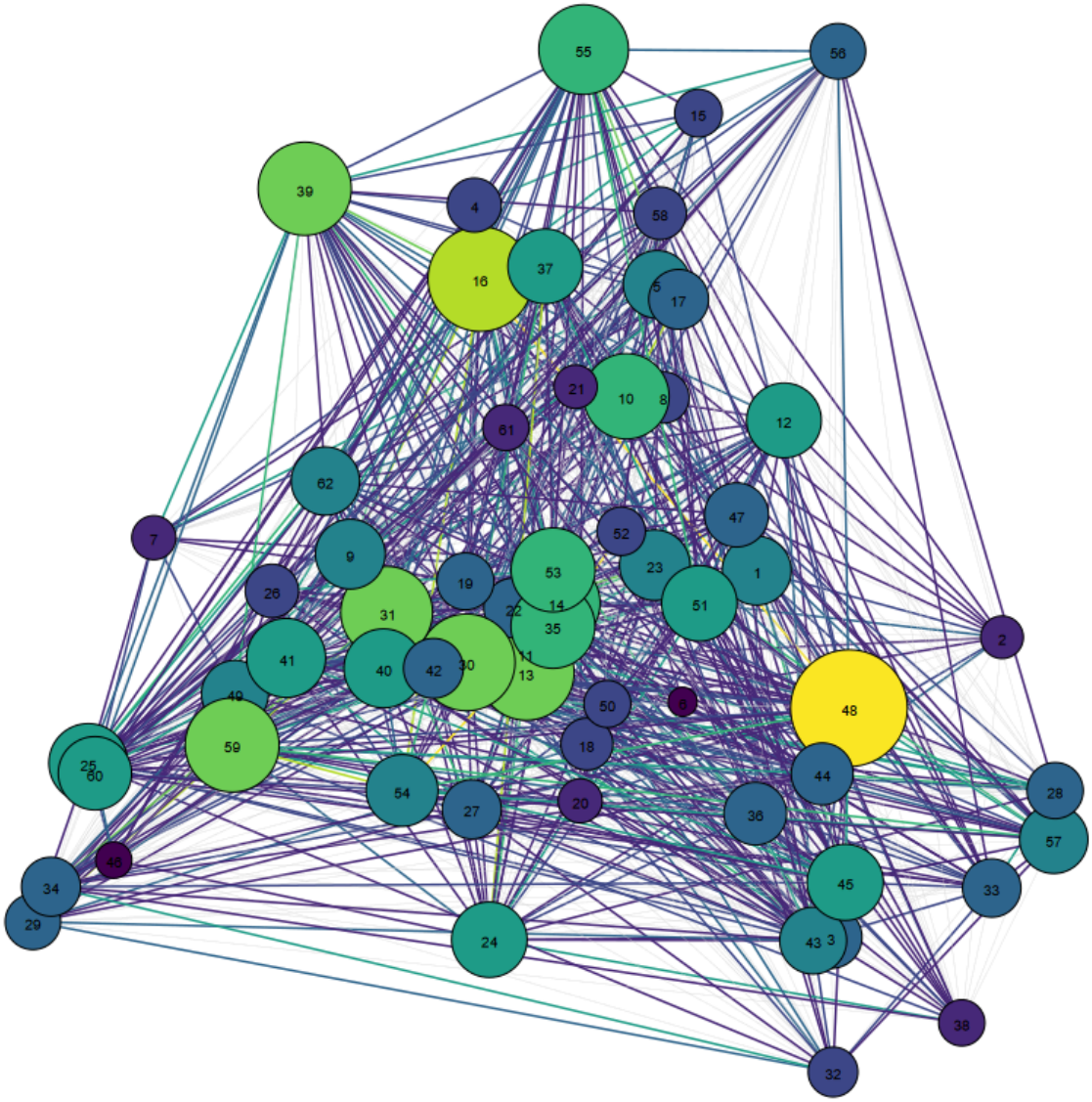
In [20]:

```
filename = "./imgs/layout_davidson_harel_simple.png"
plot_simple_graph(g, layout_type="davidson_harel", target=filename, display_img=Tru

filename = "./imgs/layout_davidson_harel.png"
plot_weighted_graph(g, layout_type="davidson_harel", target=filename, display_img=T
```

```
[✓] Skipping ./imgs/layout_davidson_harel_simple.png (already exists)
```



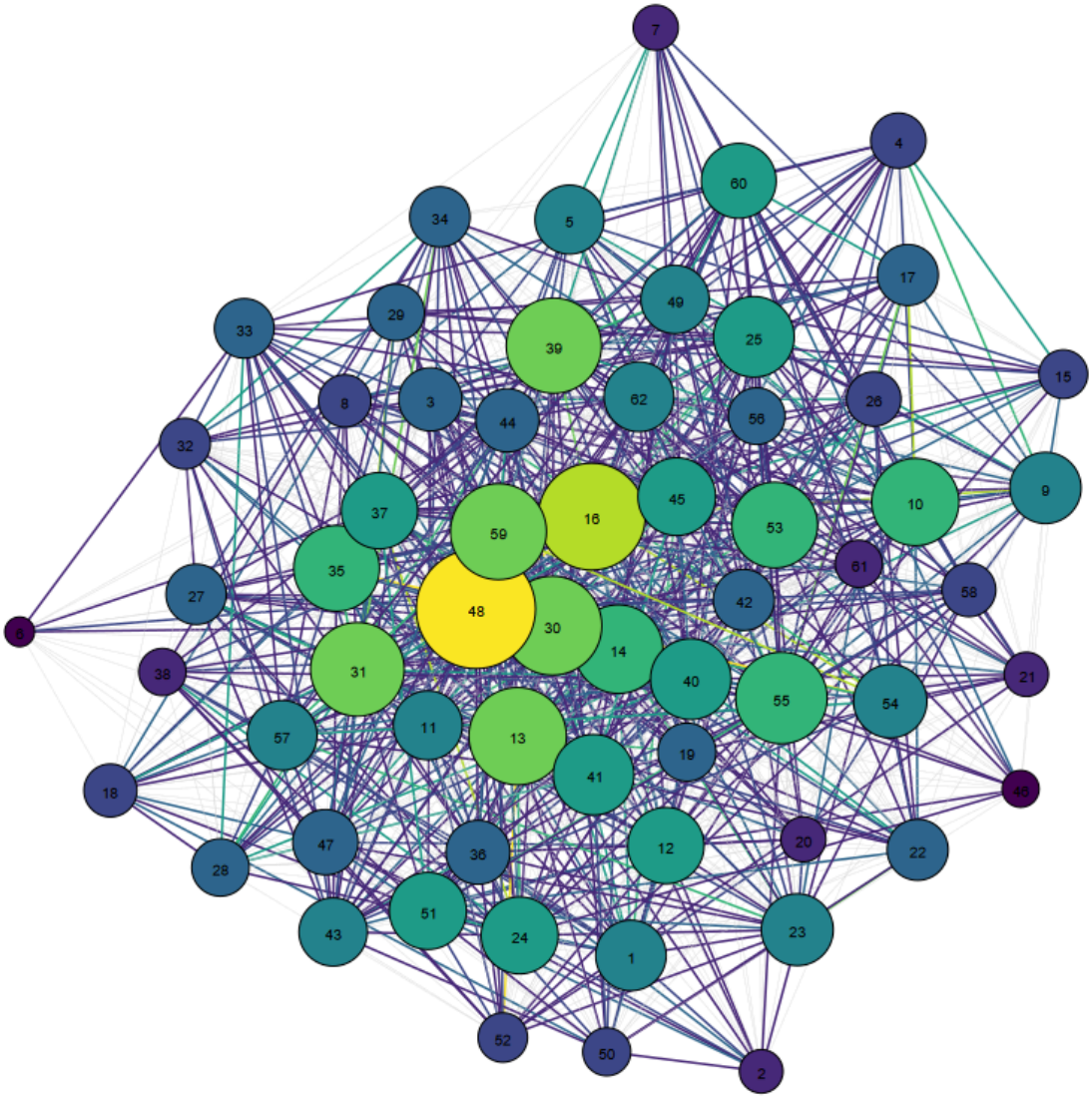
```
[✓] Skipping ./imgs/layout_davidson_harel.png (already exists)
```



Fruchterman-Reingold layout

In [21]:
```
filename = "./imgs/layout_fr.png"
plot_weighted_graph(g, layout_type="fr", target=filename, display_img=True)
```

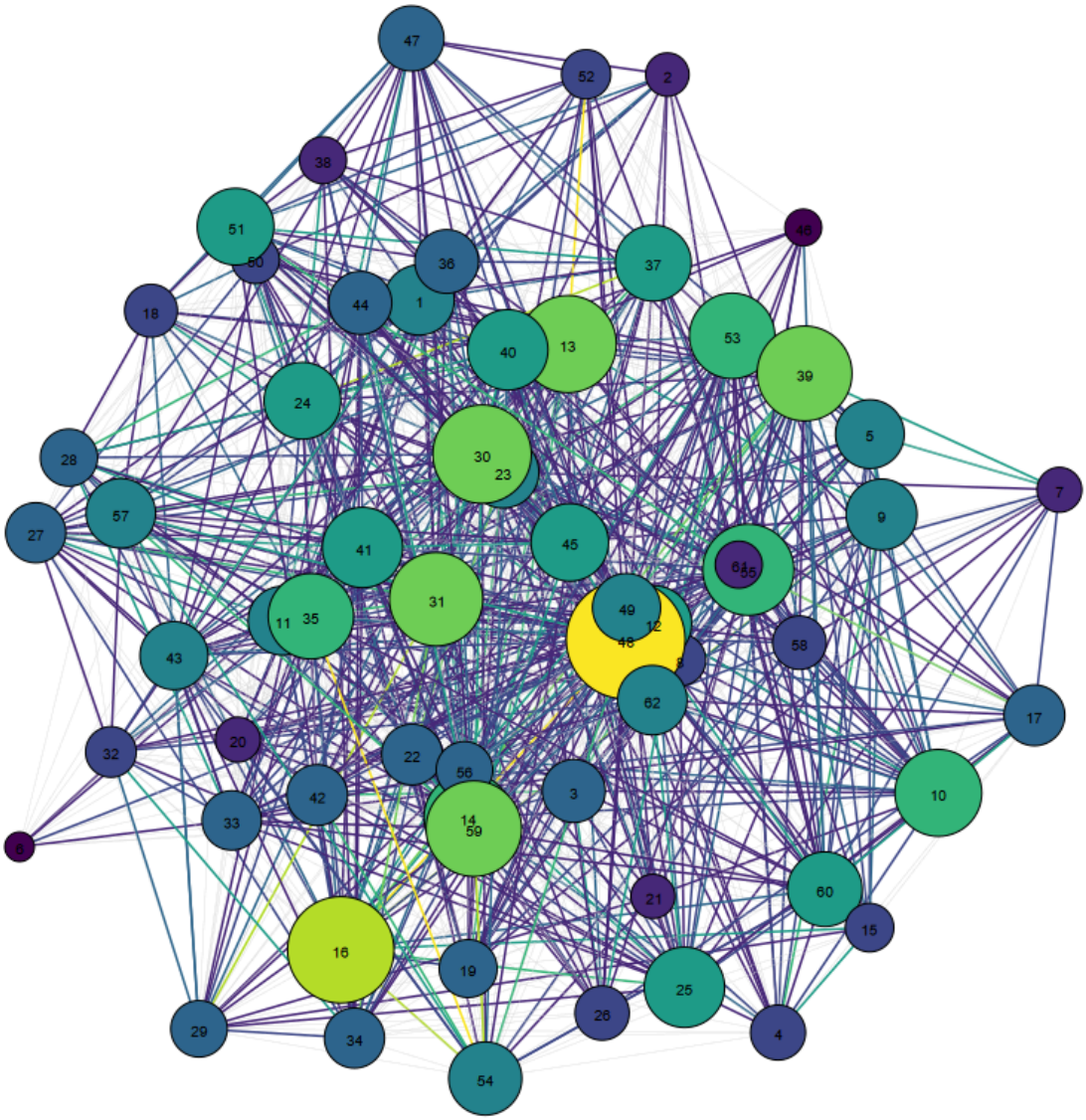
```
[✓] Skipping ./imgs/layout_fr.png (already exists)
```



Graphopt layout

In [22]:
```
filename = "./imgs/layout_graphopt.png"
plot_weighted_graph(g, layout_type="graphopt", target=filename, display_img=True)
```

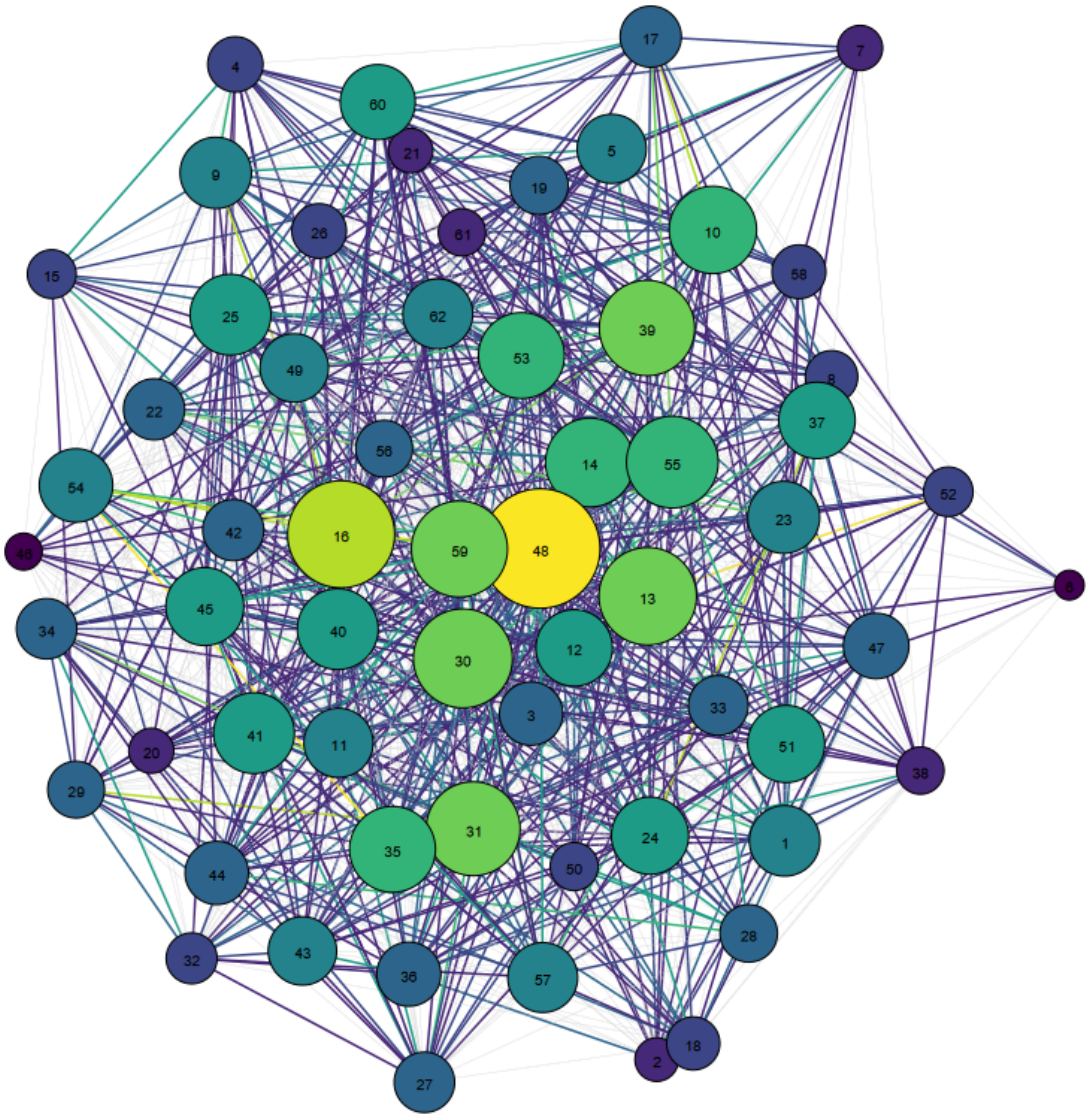
```
[✓] Skipping ./imgs/layout_graphopt.png (already exists)
```



Kamada-Kawai layout

In [23]:
```
filename = "./imgs/layout_kk.png"
plot_weighted_graph(g, layout_type="kk", target=filename, display_img=True)
```

```
[✓] Skipping ./imgs/layout_kk.png (already exists)
```



Comparison

Although we can not see much in the graph some insights can be obtained:

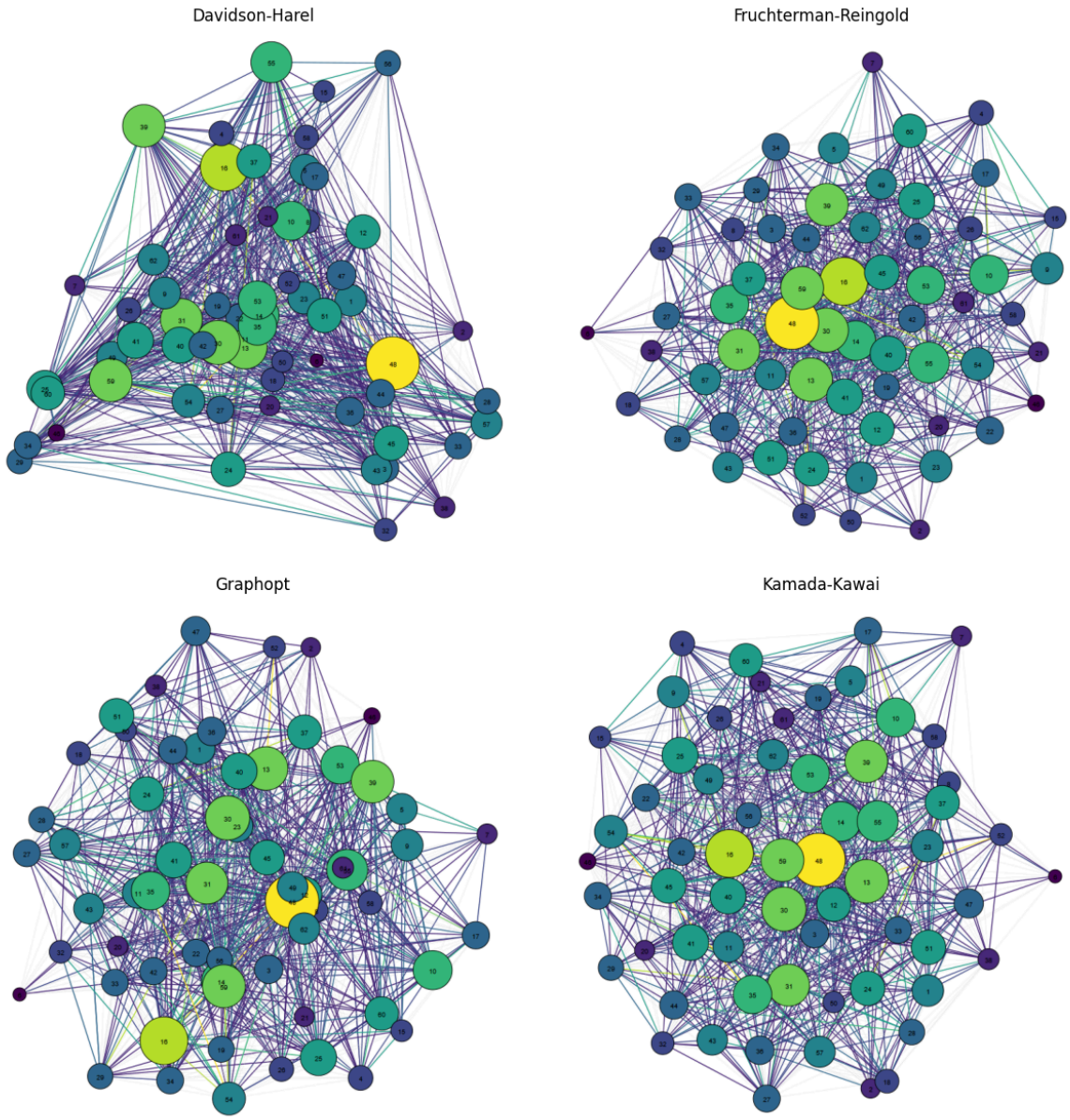
- Macaque 6 seems to have a great separation from the rest of the network in most of the layouts, except with the Davidson-Harel layout.
- Graphopt, Fruchterman-Reingold and Kamada-Kawai displays a more circular network leaving the less important vertex in the outskirts of the plot, while the other has a triangular shape separating groups of vertices, and mixing vertices of all importances in the center.
- The macaque with most importance (48) is displayed in the center with most of the layouts.

In general, all the layouts gaves similar representations of the graph, except of Davidson-Harel.

In [24]:
```
import matplotlib.pyplot as plt
from PIL import Image
```

```python
# Layouts and labels
layouts = ["davidson_harel", "fr", "graphopt", "kk"]
layout_titles = ["Davidson-Harel", "Fruchterman-Reingold", "Graphopt", "Kamada-Kawa

# Save image paths
img_paths = []

# Generate and save plots using your updated function
for layout in layouts:
    filename = f"./imgs/layout_{layout}.png"
    plot_weighted_graph(g, layout_type=layout, target=filename, display_img=False)
    img_paths.append(filename)

# Display them in a 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 12))

for ax, img_path, title in zip(axs.flat, img_paths, layout_titles):
    img = Image.open(img_path)
    ax.imshow(img)
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout()
plt.show()
```

```
[✓] Skipping ./imgs/layout_davidson_harel.png (already exists)
[✓] Skipping ./imgs/layout_fr.png (already exists)
[✓] Skipping ./imgs/layout_graphopt.png (already exists)
[✓] Skipping ./imgs/layout_kk.png (already exists)
```



Multidimensional Scaling (MDS)

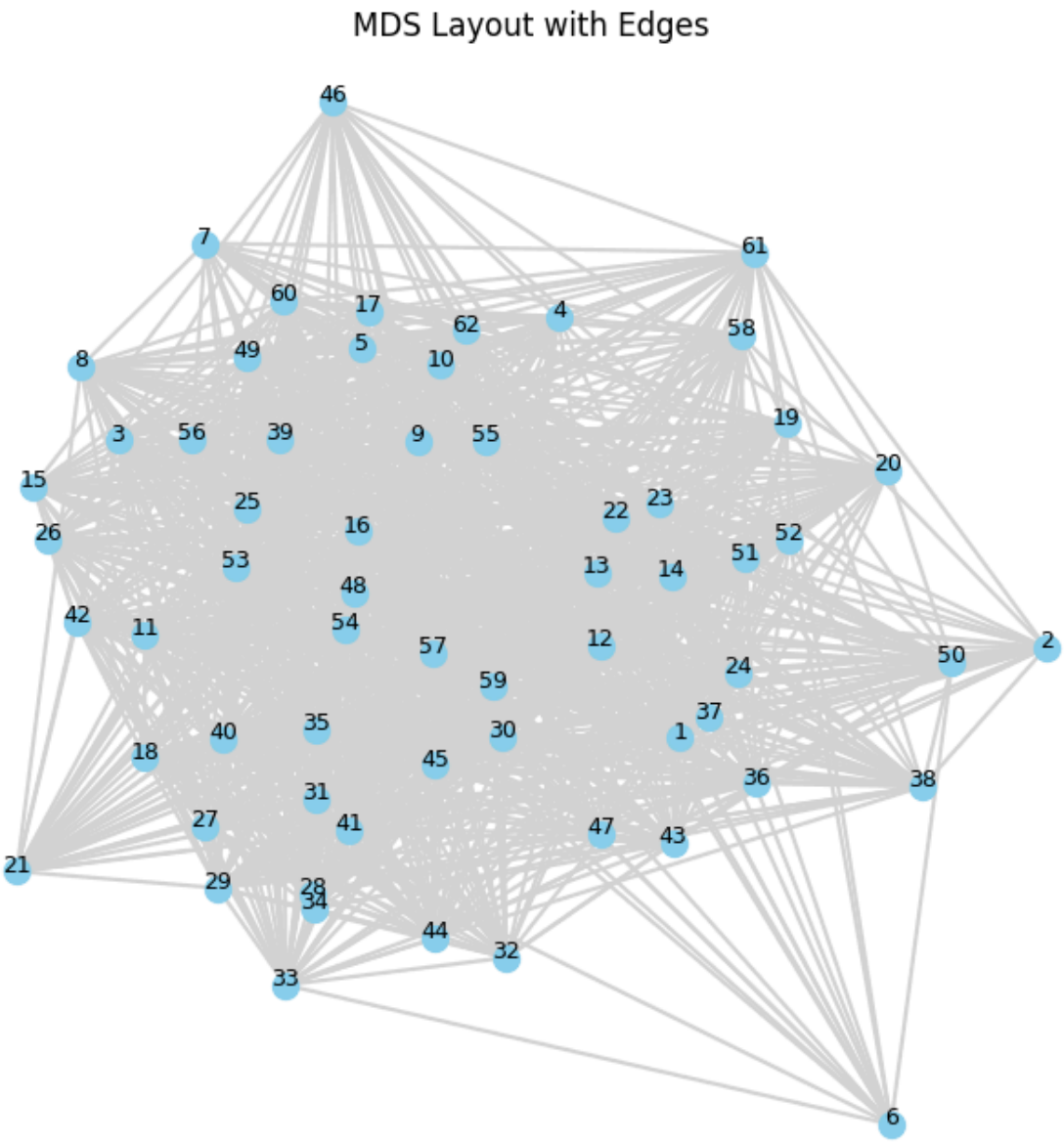
We create a 2D representation of the graph throught the MDS, where the matrix of distances are the shortest paths taking into account the inverse weights.

In [25]:
```
from sklearn.manifold import MDS
import numpy as np

# Compute the shortest path distance matrix
dist_matrix = g.shortest_paths(weights="inv_weight")

# Run MDS
mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42)
mds_coords = mds.fit_transform(dist_matrix)
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_26752\2062823136.py:5: DeprecationWarn
ing: Graph.shortest_paths() is deprecated; use Graph.distances() instead
  dist_matrix = g.shortest_paths(weights="inv_weight")
```

Initially, we made a plot without displaying the weights. The result does not seems to be more informative than what we saw before and the plot is still caothic giving not a lot of information.

In [26]:
```
plt.figure(figsize=(8, 8))

# Plot edges
for edge in g.get_edgelist():
    x0, y0 = mds_coords[edge[0]]
    x1, y1 = mds_coords[edge[1]]
    plt.plot([x0, x1], [y0, y1], color='lightgray', zorder=1)

# Plot nodes
plt.scatter(mds_coords[:, 0], mds_coords[:, 1], s=100, color='skyblue', zorder=2)

# Add labels
for i, label in enumerate(g.vs["name"]):
    plt.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center', zo

plt.title("MDS Layout with Edges")
plt.axis('off')
plt.show()
```

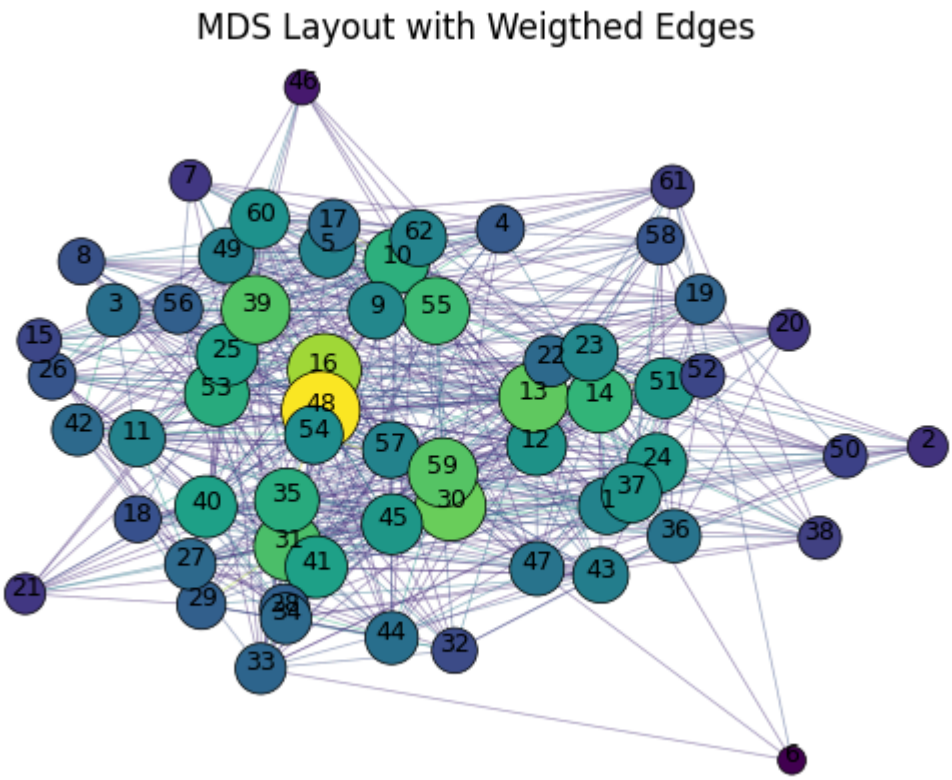


Then, we applied the color and size gradient as before, and we can see that the Multidimensional Scaling is by far the best representation that we have obtained of the network.

The most important vertex (Macaque 48) is in some way centered, and around it we can diferenciate two "rings". Closer to this node, the following most important macaques, and on the outskirts of the plot, the less important ones. Allowing to see a kind of clustering based on the dominance-relations.

In [27]:
```
weights = np.array(g.es["weight"])
inv_weights = np.array(g.es["inv_weight"])
threshold = np.percentile(weights, 60)
norm = colors.Normalize(vmin=weights.min(), vmax=weights.max())
cmap = cm.get_cmap('viridis')
print("threshold: ", threshold)
edges_to_plot = [e.index for e in g.es if e["weight"] >= threshold]

# Compute node strength from strong edges
node_strength = np.zeros(g.vcount())
for e in g.es:
    if e["weight"] >= threshold:
        node_strength[e.source] += e["weight"]
        node_strength[e.target] += e["weight"]

# Normalize node strength to size and color
min_size, max_size = 100, 800
normalized_strength = (node_strength - node_strength.min()) / (node_strength.max()
node_sizes = min_size + (max_size - min_size) * normalized_strength
node_colors = [cmap(val) for val in normalized_strength]

for e in g.es:
    if e.index not in edges_to_plot:
        continue  # Skip plotting this weak edge
    i, j = e.source, e.target
    x0, y0 = mds_coords[i]
    x1, y1 = mds_coords[j]
    w = e["weight"]
    color = cmap(norm(w))
    plt.plot([x0, x1], [y0, y1], color=color, linewidth=0.5, alpha=0.5, zorder=1)

# Plot nodes
plt.scatter(
    mds_coords[:, 0], mds_coords[:, 1],
    s=node_sizes, color=node_colors,
    edgecolor='black', linewidth=0.5, zorder=2
)

# Add labels
for i, label in enumerate(g.vs["name"]):
    plt.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center', zo

plt.title("MDS Layout with Weigthed Edges")
plt.axis('off')
plt.show()
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_26752\3662256221.py:5: MatplotlibDepre
cationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be
removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_
cmap()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap('viridis')
threshold:  2.0
```



In [28]:
```
#get_node_info(g, 6, threshold_pt=60)
```

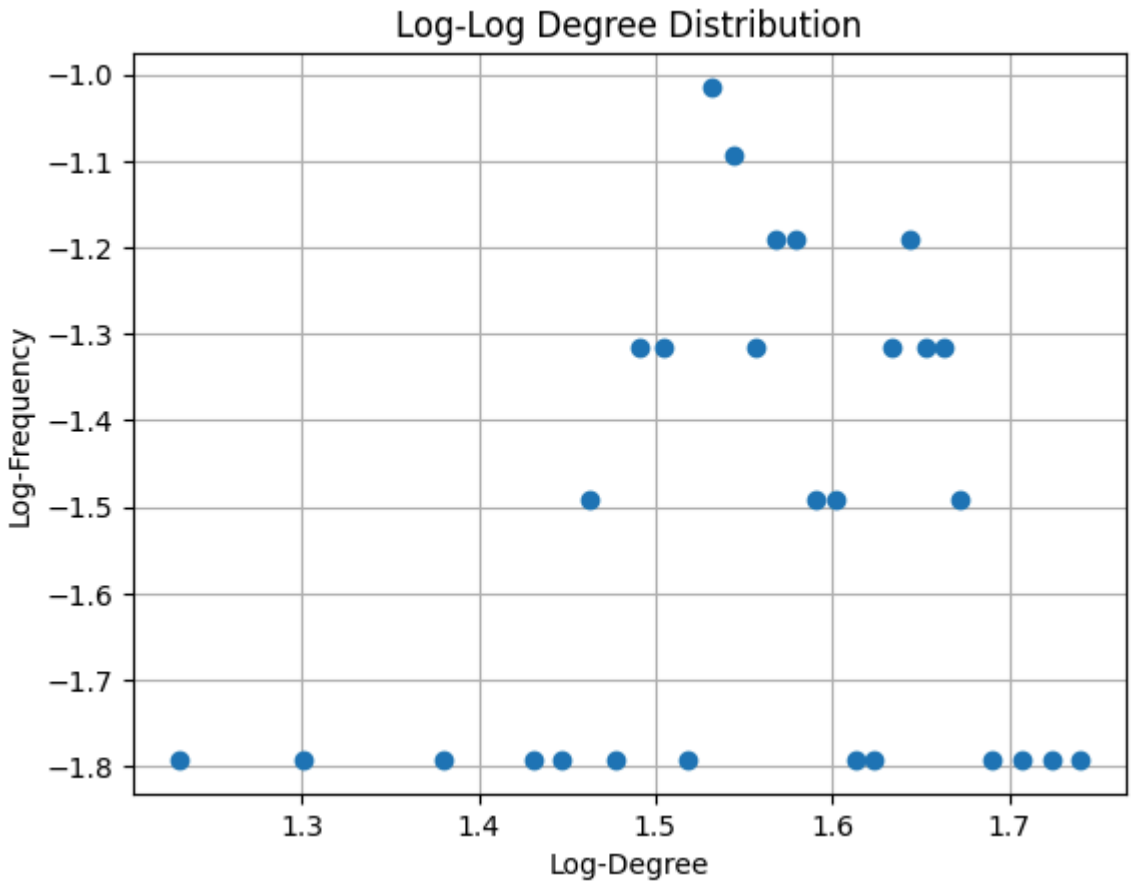
Vertex Characteristics

Degree Distribution

Unweighted log-log Degree Distribution

We clearly see that the degree distribution is not following a power law. Just for further proof, we will fit the data to a power law distribution and validate that the fit is not good.

In [29]:
```
degrees = g.degree()
unique_degrees, counts = np.unique(degrees, return_counts=True)

# Log-Log degree distribution
log_degrees = np.log10(unique_degrees[probs > 0])
log_probs = np.log10(probs[probs > 0])

plt.figure()
plt.plot(log_degrees, log_probs, 'o')
plt.xlabel("Log-Degree")
plt.ylabel("Log-Frequency")
plt.title("Log-Log Degree Distribution")
plt.grid(True)
plt.show()
```



In [30]:
```
# Clean up for fitting: remove zero-degree entries to avoid log(0)
mask = (unique_degrees > 0) & (probs > 0)
x = unique_degrees[mask]
y = probs[mask]

log_x = np.log10(x)
log_y = np.log10(y)

# Use weights = 1/x to mimic WLS emphasis on lower degrees
X = sm.add_constant(log_x)
wls_model = sm.WLS(log_y, X, weights=1/log_x)
results = wls_model.fit()

print(results.summary())
alpha = results.params[1]
print(f"\nEstimated alpha: {alpha:.4f}")
```

```
                            WLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.049
Model:                            WLS   Adj. R-squared:                  0.013
Method:                 Least Squares   F-statistic:                     1.353
Date:                Mon, 28 Apr 2025   Prob (F-statistic):              0.255
Time:                        22:01:15   Log-Likelihood:                -2.0220
No. Observations:                  28   AIC:                             8.044
Df Residuals:                      26   BIC:                             10.71
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.2605      0.626     -3.610      0.001      -3.547      -0.973
x1             0.4694      0.404      1.163      0.255      -0.360       1.299
==============================================================================
Omnibus:                        5.630   Durbin-Watson:                   0.997
Prob(Omnibus):                  0.060   Jarque-Bera (JB):                2.134
Skew:                           0.299   Prob(JB):                        0.344
Kurtosis:                       1.787   Cond. No.                         27.0
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly s
pecified.

Estimated alpha: 0.4694
```

As we can see, the R-squared is 0.0494 and the adjusted R-squared is 0.0129, which means that the model does not fit the data well.

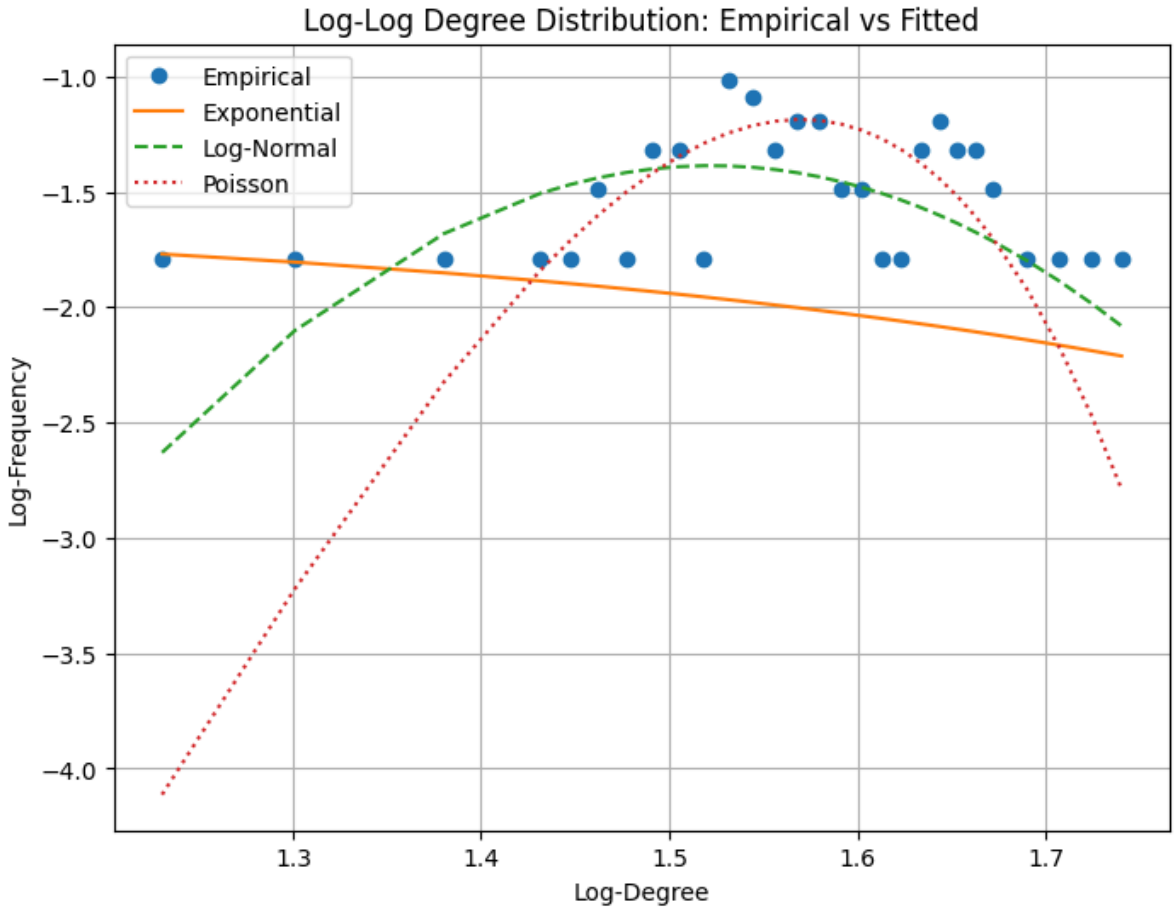
In [31]:
```python
r2 = results.rsquared
print(f"R-squared: {r2:.4f}")
adj_r2 = results.rsquared_adj
print(f"Adjusted R-squared: {adj_r2:.4f}")
```

```
R-squared: 0.0494
Adjusted R-squared: 0.0129
```

We tried to plot the data on other distributions, but the results were not good either.

In [32]:
```python
# Exponential
exp_loc, exp_scale = expon.fit(x, floc=0)
pdf_exp = expon.pdf(x, loc=exp_loc, scale=exp_scale)

# Log-Normal
ln_shape, ln_loc, ln_scale = lognorm.fit(x, floc=0)
pdf_ln = lognorm.pdf(x, ln_shape, loc=ln_loc, scale=ln_scale)

# Poisson (discrete, use PMF)
mu = np.mean(degrees)
pmf_pois = poisson.pmf(x, mu)

# Step 3: Log-transform fitted values
log_pdf_exp = np.log10(pdf_exp)
log_pdf_ln = np.log10(pdf_ln)
log_pmf_pois = np.log10(pmf_pois)

# Step 4: Plot
plt.figure(figsize=(8, 6))
plt.plot(log_x, log_y, 'o', label="Empirical")
plt.plot(log_x, log_pdf_exp, '-', label="Exponential")
```

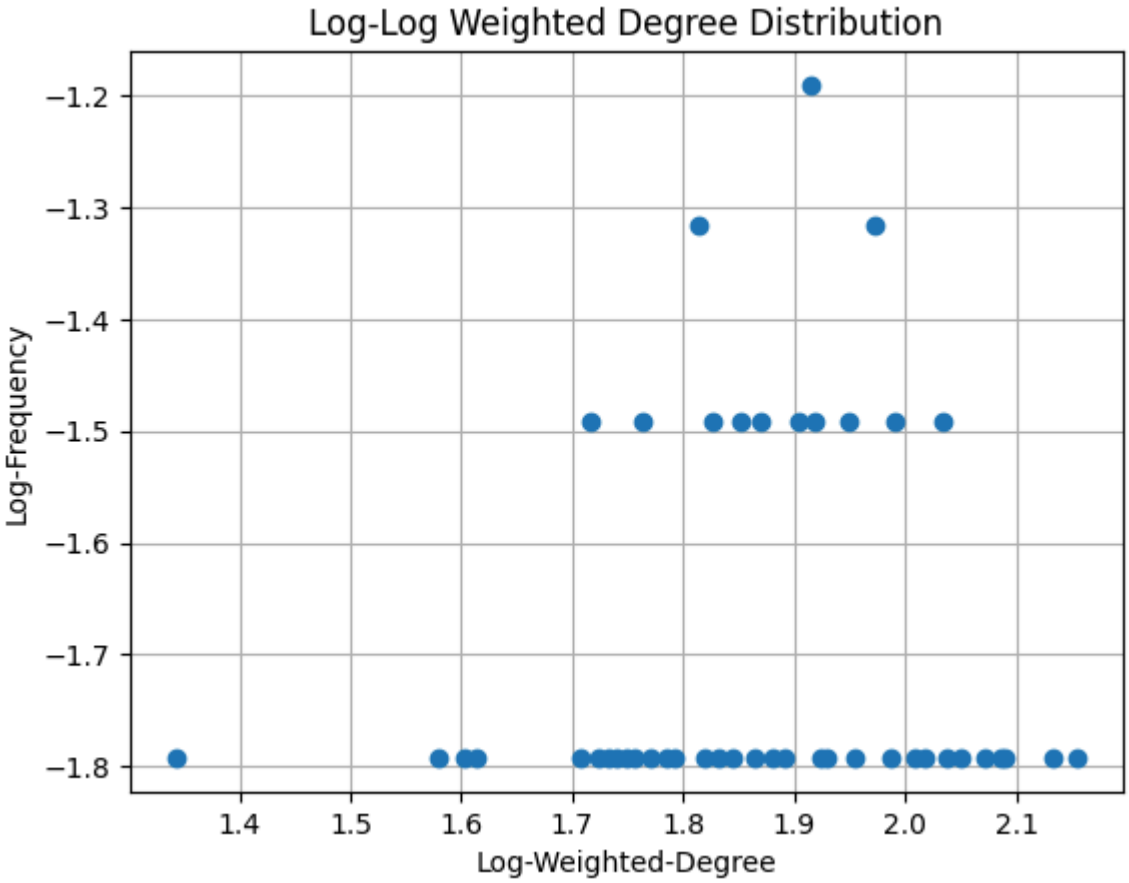
```
plt.plot(log_x, log_pdf_ln, '--', label="Log-Normal")
plt.plot(log_x, log_pmf_pois, ':', label="Poisson")
plt.xlabel("Log-Degree")
plt.ylabel("Log-Frequency")
plt.title("Log-Log Degree Distribution: Empirical vs Fitted")
plt.legend()
plt.grid(True)
plt.show()
```



Weighted log-log Degree Distribution

We can see that the weighted degree distribution is not following a power law either. Just by looking at the plot we can see that the data is not following a straight line and the shape is similar to the unweighted one, so we won't compute the fit for this one.

In [33]:
```
weighted_degrees = g.strength(weights="weight")
unique_w_deg, w_counts = np.unique(weighted_degrees, return_counts=True)
w_probs = w_counts / sum(w_counts)

# Log-log weighted degree distribution
log_w_deg = np.log10(unique_w_deg[w_probs > 0])
log_w_probs = np.log10(w_probs[w_probs > 0])

plt.figure()
plt.plot(log_w_deg, log_w_probs, 'o')
plt.xlabel("Log-Weighted-Degree")
plt.ylabel("Log-Frequency")
plt.title("Log-Log Weighted Degree Distribution")
plt.grid(True)
plt.show()
```



Centrality Measures

In [34]:
```python
def top_k_centrality(centrality, label, k=5):
    top_indices = np.argsort(centrality)[::-1][:k]
    print(f"\n 🔹 Top {k} nodes by {label}:")
    for idx in top_indices:
        print(f"{g.vs[idx]['name']}: {centrality[idx]:.4f}")
```

Degree Centrality

The most central vertices according to the degree centrality are the ones with most connections. On the weighted case, the most central is the one with most connections taking into account the weights. In both cases, the most central vertex is the 48. We can check that in the previous section, on the graphs plots, the top vertices are the same. (As their size and color in the plot depend on the weights)

In [35]:
```python
# Unweighted degree
degrees = np.array(g.degree())
top_unweighted = np.argsort(degrees)[::-1]  # descending order


# Weighted degree (strength)
weighted_degrees = np.array(g.strength(weights="weight"))
top_weighted = np.argsort(weighted_degrees)[::-1]

# How many top nodes to show
k = 5

print(" 🔹 Top vertices by unweighted degree:")
for idx in top_unweighted[:k]:
```

```
    print(f"{g.vs[idx]['name']}: degree = {degrees[idx]}")

print("\n 🔸 Top vertices by weighted degree:")
for idx in top_weighted[:k]:
    print(f"{g.vs[idx]['name']}: weighted degree = {weighted_degrees[idx]}")
```

```
 🔹 Top vertices by unweighted degree:
48: degree = 55
30: degree = 53
59: degree = 51
12: degree = 49
40: degree = 47

 🔸 Top vertices by weighted degree:
48: weighted degree = 143.0
16: weighted degree = 136.0
59: weighted degree = 123.0
30: weighted degree = 122.0
13: weighted degree = 118.0
```

As we can see on the degree distribution plotted at the beginning, the most connected vertices have a significantly higher degree than the rest of the vertices. This means that they are more important in the network, and that they are more likely to be connected to other important vertices. We can check that by computing the z-scores of the top vertices. The z-score is a measure of how many standard deviations a value is from the mean. A high z-score means that the vertex is more important than the average vertex in the network.

In our case the top 5 nodes by unweighted degree have z-scores ranging from 1.26 to 2.34, with the highest degree node reaching 2.34 standard deviations above the mean. These values suggest that while the top nodes are more central than average, they are not extreme outliers. The z-scores are moderately high, indicating a moderate centrality hierarchy but not a strong dominance by a few nodes. There's a gradient of centrality rather than a sharp concentration.

For the weighted degree (strength), the top 5 nodes show z-scores between 1.64 and 2.67, which are consistently higher than those of the unweighted case. The top node, with a z-score of 2.67, is particularly notable — this suggests it is substantially more central in terms of accumulated edge weight. Overall, weighted centrality shows a more pronounced disparity, implying that some nodes are involved in stronger or more frequent interactions than others, even if the number of neighbors isn't dramatically different.

In [36]:
```
degrees = np.array(g.degree())
top_k = 5
sorted_deg = np.sort(degrees)[::-1]
top = sorted_deg[:top_k]

mean_deg = degrees.mean()
std_deg = degrees.std()
print("&"*50)
print("Top degrees:", top)
print(f"Mean: {mean_deg:.2f}, Std: {std_deg:.2f}")
z_scores = (top - mean_deg) / std_deg
print("Z-scores of top nodes:", z_scores)
print("&"*50)

weighted_degrees = np.array(g.strength(weights="weight"))
sorted_weighted_deg = np.sort(weighted_degrees)[::-1]
top_weighted = sorted_weighted_deg[:top_k]
```

```
mean_weighted_deg = weighted_degrees.mean()
std_weighted_deg = weighted_degrees.std()
print("\n")
print("&"*50)
print("Top weighted degrees:", top_weighted)
print(f"Mean: {mean_weighted_deg:.2f}, Std: {std_weighted_deg:.2f}")
z_scores_weighted = (top_weighted - mean_weighted_deg) / std_weighted_deg
print("Z-scores of top nodes (weighted):", z_scores_weighted)
print("&"*50)
```

```
&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&
Top degrees: [55 53 51 49 47]
Mean: 37.65, Std: 7.40
Z-scores of top nodes: [2.34487777 2.07465022 1.80442267 1.53419512 1.26396757]
&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&


&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&
Top weighted degrees: [143. 136. 123. 122. 118.]
Mean: 78.55, Std: 24.11
Z-scores of top nodes (weighted): [2.67363053 2.38325124 1.84397541 1.80249266 1.6
3656163]
&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&
```

Closeness Centrality

The closeness centrality is a measure of how close a vertex is to all other vertices in the network. The most central vertex is the one that can reach all other vertices in the shortest time.

In the unweighted case, node 48 is the most central, suggesting it has short paths to many other macaques in terms of connection steps. However, when accounting for the number of dominance interactions (i.e., using inverse interaction frequency as distance), node 16 becomes the most central. This indicates that node 16, and others like 54 and 13, are not just well connected, they are connected through frequent dominance interactions, placing them strategically at the core of high-intensity interaction clusters.

In [37]:
```
# --- Closeness centrality ---
closeness_unw = np.array(g.closeness(normalized=True))
closeness_w = np.array(g.closeness(weights="inv_weight", normalized=True))

top_k_centrality(closeness_unw, "Closeness (unweighted)")
top_k_centrality(closeness_w, "Closeness (weighted)")
```

```
🔹 Top 5 nodes by Closeness (unweighted):
48: 0.9104
30: 0.8841
59: 0.8592
12: 0.8356
40: 0.8133

🔹 Top 5 nodes by Closeness (weighted):
16: 2.7633
54: 2.5946
59: 2.5703
48: 2.5627
13: 2.4766
```

Betweenness Centrality

Betweenness centrality is a measure of how often a vertex acts as a bridge along the shortest path between two other vertices. The most central vertex is the one that lies on the most shortest paths between other vertices.

Node 48 also leads in unweighted betweenness, suggesting it may act as a bridge between groups in the macaque social structure. However, in the weighted analysis, node 16 dominates. This means node 16 frequently lies along the shortest, most interaction-rich paths between other individuals.

In a social dominance context, this could point to central individuals involved in resolving or mediating dominance dynamics, possibly respected or high-status macaques based on the frequency and breadth of their dominance interactions.

In [38]:
```
n = g.vcount()

# --- Betweenness centrality ---
betweenness_unw = np.array(g.betweenness())
betweenness_unw = betweenness_unw / ((n - 1) * (n - 2) / 2)

betweenness_w = np.array(g.betweenness(weights="inv_weight"))
betweenness_w = betweenness_w / ((n - 1) * (n - 2) / 2)

top_k_centrality(betweenness_unw, "Betweenness (unweighted)")
top_k_centrality(betweenness_w, "Betweenness (weighted)")
```

```
🔹 Top 5 nodes by Betweenness (unweighted):
48: 0.0166
30: 0.0139
59: 0.0126
12: 0.0119
55: 0.0112

🔹 Top 5 nodes by Betweenness (weighted):
16: 0.1229
13: 0.0839
48: 0.0615
59: 0.0607
54: 0.0552
```

Eigenvector Centrality

Eigenvector centrality is a measure of the influence of a node in a network. It considers not just the number of connections a node has, but also the quality and influence of those connections. A node is considered important if it is connected to other important nodes.

In the unweighted case, node 48 stands out as the most "influential" macaque, followed closely by nodes 30 and 59, likely forming a core subgroup in the dominance network.

When accounting for interaction strength (weighted), node 16 surpasses 48, suggesting that 16 may not only be connected to central individuals, but those connections are reinforced by frequent dominance interactions.

This shift highlights how interaction intensity reveals hidden social influence, not captured by topology alone.

In [39]:
```
# --- Eigenvector centrality ---
eigen_unw = np.array(g.eigenvector_centrality())
eigen_w = np.array(g.eigenvector_centrality(weights="weight"))
top_k_centrality(eigen_unw, "Eigenvector (unweighted)")
top_k_centrality(eigen_w, "Eigenvector (weighted)")
```

```
🔹 Top 5 nodes by Eigenvector (unweighted):
48: 1.0000
30: 0.9768
59: 0.9403
12: 0.9002
16: 0.8801

🔹 Top 5 nodes by Eigenvector (weighted):
16: 1.0000
48: 0.9999
59: 0.8540
30: 0.8478
13: 0.8069
```

PageRank centrality

PageRank centrality is a measure of the importance of a node in a network, based on the idea that important nodes are likely to be connected to other important nodes.

Node 48 consistently ranks highest in both weighted and unweighted PageRank, confirming its prominent role in the dominance network. With weights included, node 16 also emerges near the top, reinforcing the idea that frequency of interactions elevates its perceived social standing.

The alignment between PageRank and eigenvector results supports a stable, central core group, with node 48 as a persistent central figure and node 16 gaining importance when intensity is factored in.

In [40]:
```
# --- PageRank ---
pagerank_unw = np.array(g.pagerank())
pagerank_w = np.array(g.pagerank(weights="weight"))

top_k_centrality(pagerank_unw, "PageRank (unweighted)")
top_k_centrality(pagerank_w, "PageRank (weighted)")
```

```
🔹 Top 5 nodes by PageRank (unweighted):
48: 0.0226
30: 0.0218
59: 0.0210
12: 0.0203
40: 0.0196

🔹 Top 5 nodes by PageRank (weighted):
48: 0.0274
16: 0.0259
59: 0.0239
30: 0.0238
13: 0.0232
```

Edge Characteristics

In [41]:
```
def top_k_edges(edge_btw, label, k=5):
    top_indices = np.argsort(edge_btw)[::-1][:k]
    print(f"\n 🔹 Top {k} edges by {label}:")
    for idx in top_indices:
        e = g.es[idx]
        v1, v2 = g.vs[e.source]["name"], g.vs[e.target]["name"]
        print(f"{v1} - {v2}: {edge_btw[idx]:.4f}")
```

Edge Betweenness Centrality

Edge betweenness centrality is a measure of how often an edge lies on the shortest path between two other vertices. The most central edge is the one that lies on the most shortest paths between other vertices.

The top edges (e.g., between nodes 48–6, 48–7, and 30–6) appear to serve as structural bridges in the macaque dominance network. These edges lie on many of the shortest paths and likely connect dense subgroups or clusters, making them essential for maintaining overall network connectivity.

Their high centrality suggests that dominance interactions between these individuals are critical for the indirect relational structure, even if the interactions aren't especially frequent.

When accounting for the number of interactions (using inv_weight as distance), a different set of edges emerges, notably 59–54, 16–48, and 35–54. These links carry significant interaction volume while still connecting crucial paths.

This indicates that not only do these edges bridge important network regions, but they also do so via frequent dominance interactions, which could point to key pathways in reinforcing or mediating social structure through repetition and intensity.

In [42]:
```
# Unweighted edge betweenness
edge_btw_unw = np.array(g.edge_betweenness())

# Weighted edge betweenness using inverse weight
edge_btw_w = np.array(g.edge_betweenness(weights="inv_weight"))

top_k_edges(edge_btw_unw, "Edge Betweenness (unweighted)")
top_k_edges(edge_btw_w, "Edge Betweenness (weighted)")
```

```
 🔹 Top 5 edges by Edge Betweenness (unweighted):
48 - 6: 4.7522
48 - 7: 4.6561
30 - 6: 4.6432
61 - 6: 4.2829
12 - 6: 4.0198

 🔹 Top 5 edges by Edge Betweenness (weighted):
59 - 54: 65.0000
16 - 48: 60.0000
35 - 54: 58.0000
16 - 9: 57.5000
16 - 54: 54.0000
```

While nodes 16 and 48 consistently appear as the most central individuals across multiple centrality measures, we cannot conclude whether they are dominant or subordinate, since the network is undirected and lacks information about interaction direction. Their high

centrality indicates they are frequent and structurally important participants in dominance interactions, and the edge connecting them likely represents a key social relationship, potentially reflecting a complex or contested dominance dynamic between two influential individuals.